



Administracion de Tecnologias de Informacion

Bases de Datos
Proyecto#1

Jacqueline Tatiana Solis Cespedes

Elaborado por:
Edwin Mauricio Membreño Artavia

II Semestre, 2026

Tabla de Contenido

Matriz de Trazabilidad..... 3

Modelo Ampliado.....5

 Modelo Chen.....5

 Modelo Lógico.....6

 Modelo Físico.....6

 DDL.....7

Justificación del Diseño del Modelo Físico (DDL) y Vistas Analíticas..... 9

Evidencias Visuales de la Interfaz Gráfica..... 10

 Módulo Gestor de Ubicaciones.....10

 Módulo Tareas Asociadas a Eventos.....12

 Módulo Disponibilidad de Usuarios y Gestión de Tiempos.....14

Anexo IA..... 16

Matriz de Trazabilidad

Gestor de ubicaciones			
Req	Fase	Tareas	Estado
RF-08	1. Modelo Conceptual	Definir la entidad ubicaciones y sus atributos obligatorios desde la interfaz: id_ubicacion (PK), nombre, direccion, ciudad y capacidad.	Completada
	2. Modelo Lógico	Estructurar la tabla relacional ubicaciones especificando tipos de datos y su clave primaria para la administración desde la GUI.	Completada
	3. Modelo Físico	Codificar el comando CREATE TABLE ubicaciones en PostgreSQL asegurando restricciones de nulidad y tipos óptimos.	Completada
	4. Interfaz Gráfica (GUI)	Desarrollar la pestaña y tabla de ubicaciones junto al formulario de entradas para registrar, consultar, actualizar y eliminar recintos.	Completada
RF-09	1. Modelo Conceptual	Establecer la relación de cardinalidad entre la entidad Eventos y la entidad ubicaciones mediante la clave foránea id_ubicacion.	Completada
	2. Modelo Lógico	Incorporar el atributo id_ubicacion como Clave Foránea (FK) en el esquema relacional de eventos para estructurar consultas complejas.	Completada
	3. Modelo Físico	Ejecutar la sentencia en PostgreSQL para añadir la relación referencial y soportar la detección de eventos simultáneos en un mismo recinto.	Completada
	4. Interfaz Gráfica (GUI)	Integrar consultas y selectores en la interfaz para consultar el histórico por ubicación y prevenir conflictos de espacio temporal.	Completada
RF-10	3. Modelo Físico	Diseñar y escribir las consultas para calcular el ranking de recintos más solicitados y con mayor volumen de eventos.	Completada
	4. Interfaz Gráfica (GUI)	Implementar un panel de reportes de solo lectura en la GUI para mostrar la tasa de uso de los espacios y la demanda operativa.	Completada

Disponibilidad de Usuarios y Gestor de Tiempos			
Req	Fase	Tareas	Estado
RF-11	1. Modelo Conceptual	Definir la entidad disponibilidades vinculada al perfil del usuario y el catálogo	Completada

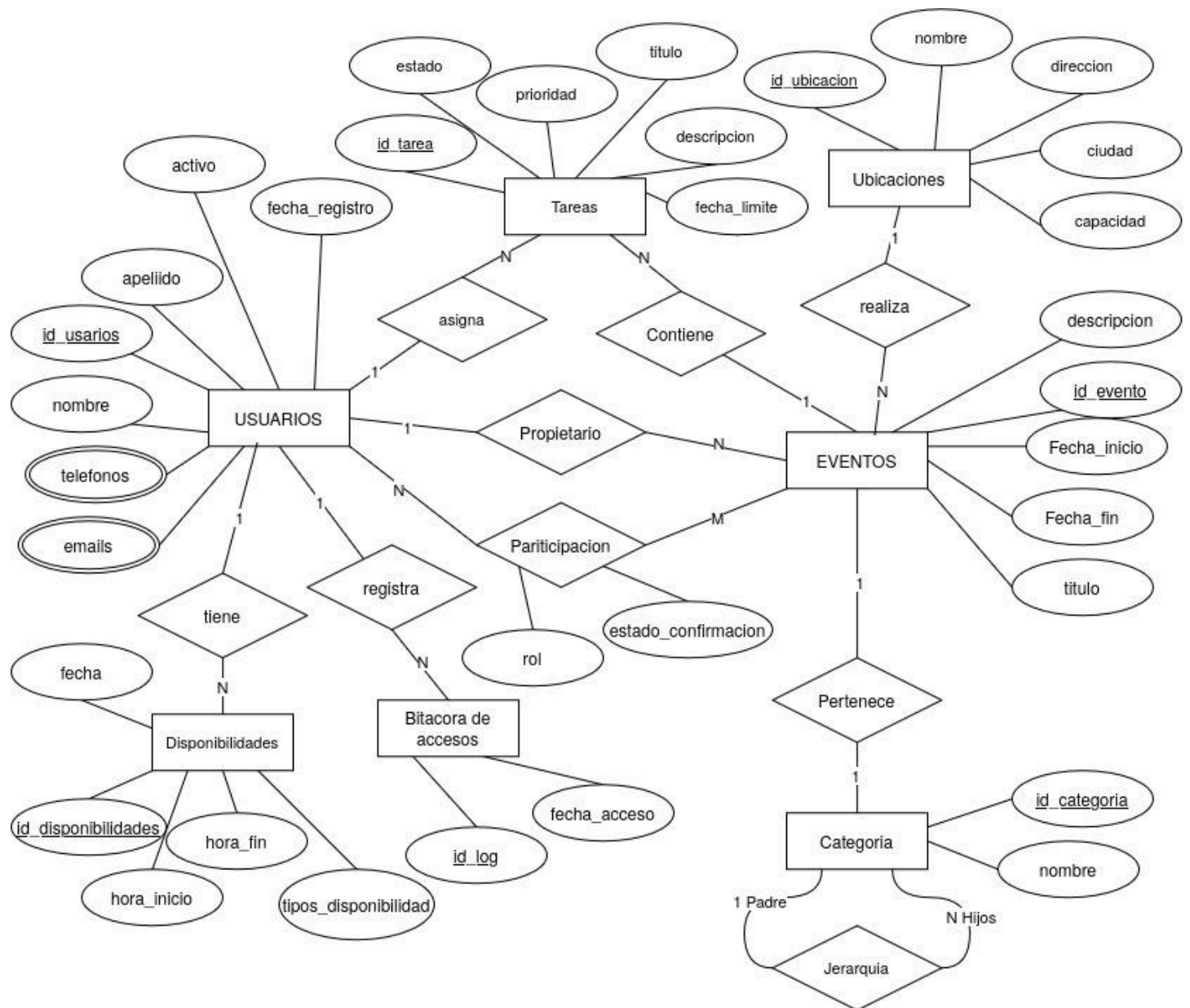
		tipos_disponibilidad con sus atributos obligatorios de fecha y horas.	
	2. Modelo Lógico	Estructurar las tablas relacionales para el control de disponibilidades y su clave foránea hacia usuarios.	Completada
	3. Modelo Físico	Codificar la creación de tablas y catálogos en PostgreSQL asegurando la integridad de las franjas temporales.	Completada
	4. Interfaz Gráfica (GUI)	Desarrollar el módulo en la interfaz para que los usuarios administren sus periodos libres u ocupados mediante operaciones CRUD completas.	Completada
RF-12	3. Modelo Físico	Estructurar las consultas analíticas en el motor de base de datos para la detección de cruces o traslapes temporales entre eventos y disponibilidades.	Completada
	4. Interfaz Gráfica (GUI)	Integrar las salidas analíticas en la GUI para determinar automáticamente usuarios libres y disponibles en un rango horario específico.	Completada

Tareas Asociadas a Eventos			
Req	Fase	Tareas	Estado
RF-15	1. Modelo Conceptual	Definir la entidad tareas vinculada a eventos con atributos de título, descripción, prioridad, fecha límite, responsable y estados.	Completada
	2. Modelo Lógico	Establecer las claves foráneas que relacionan cada tarea con su evento y usuario responsable.	Completada
	3. Modelo Físico	Ejecutar la creación de la tabla tareas en PostgreSQL con restricciones para los estados predefinidos.	Completada
	4. Interfaz Gráfica (GUI)	Habilitar formularios de control para que cada usuario realice un CRUD completo (crear, leer, actualizar estados y eliminar) desde la interfaz.	Completada
RF-16	3. Modelo Físico	Desarrollar las consultas orientadas a medir el rendimiento, tareas pendientes por usuario y detección de pendientes vencidos fuera de plazo.	Completada
	4. Interfaz Gráfica (GUI)	Configurar el despliegue de estas métricas operativas directamente en las pantallas de la aplicación.	Completada

RF-17	3. Modelo Físico	Estructurar la consulta del reporte analítico cuantitativo para filtrar tareas activas por estado y plazos vencidos.	Completada
	4. Interfaz Gráfica (GUI)	Integrar el reporte de seguimiento dentro de la interfaz del módulo de tareas para la detección temprana de sobrecargas de trabajo.	Completada

Modelo Ampliado

Modelo Chen



Modelo Lógico

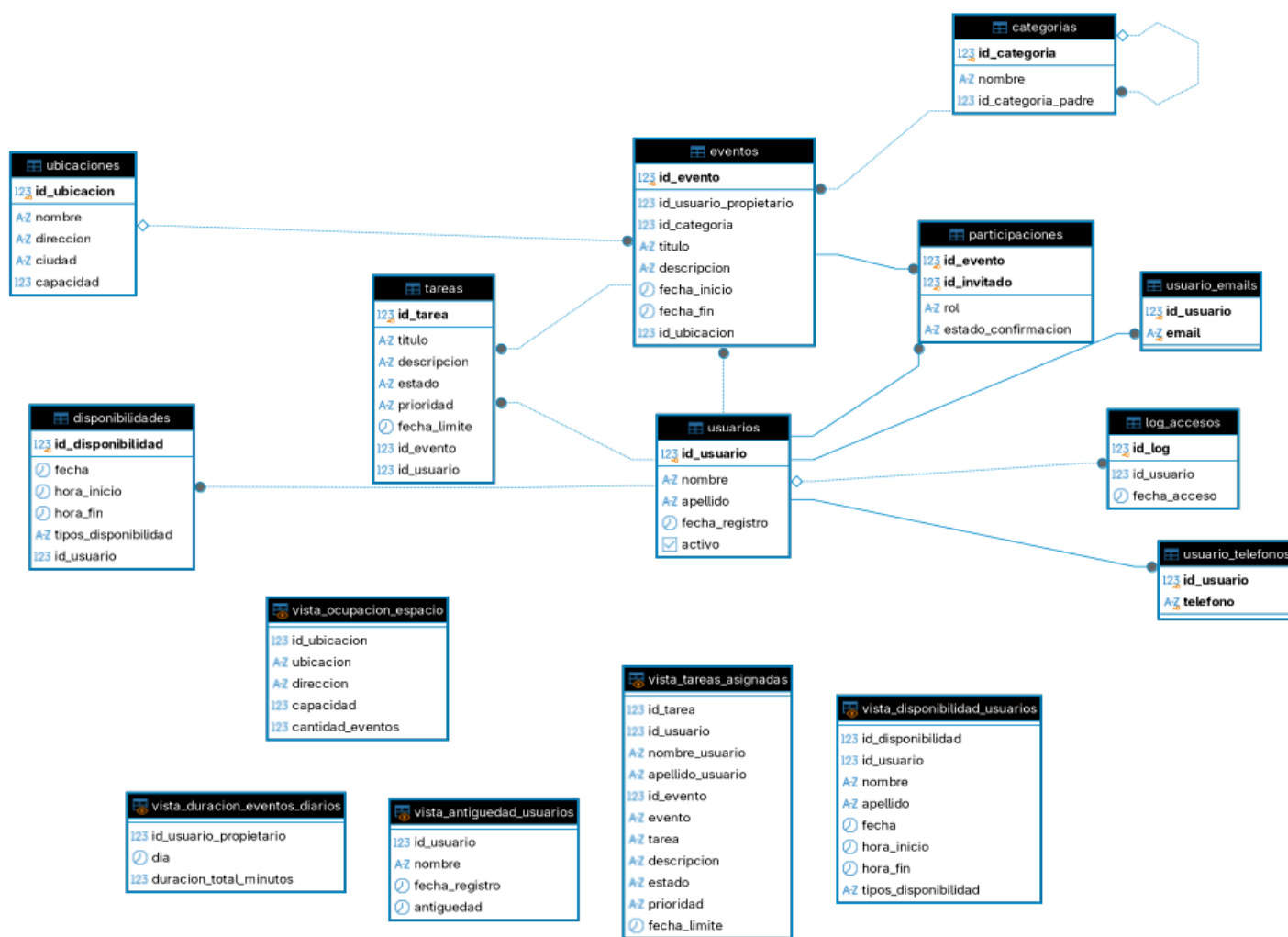
UBICACIONES (id_ubicacion [PK], nombre, direccion, ciudad, capacidad)

EVENTOS (id_evento [PK], titulo, descripcion, fecha_inicio, fecha_fin, id_usuario_propietario [FK], id_categoria [FK], id_ubicacion [FK])

TAREAS (id_tarea [PK], id_evento [FK], id_tarea_usuario [FK], titulo, descripcion, prioridad, fecha_limite, estado)

DISPONIBILIDADES (id_disponibilidad [PK], id_usuario [FK], fecha, hora_inicio, hora_fin, tipos_disponibilidad)

Modelo Físico



DDL

```
create table ubicaciones(
    id_ubicacion serial primary key,
    nombre varchar(30) not null,
    direccion varchar(50) not null,
    ciudad varchar(30) not null,
    capacidad smallint not null check (capacidad > 0)
);
create table disponibilidades (
    id_disponibilidad serial primary key,
    fecha date not null,
    hora_inicio time not null,
    hora_fin time not null,
    tipos_disponibilidad varchar(15) not null,
    id_usuario int not null,

    constraint fk_usuario_disponible
        foreign key (id_usuario)
        references usuarios(id_usuario),

    constraint chk_tipo_dispo
        check (tipos_disponibilidad in ('Disponible', 'Ocupado', 'No disponible')),

    constraint chk_horas
        check (hora_inicio < hora_fin)
);
create table tareas(
    id_tarea serial primary key,
    titulo varchar(150) not null,
    descripcion text not null,
    estado varchar(15) default 'Pendiente' check (estado in ('Pendiente', 'En progreso', 'Completada',
'Cancelada')),
    prioridad varchar(10) default 'Media' check (prioridad in ('Baja', 'Media', 'Alta')),
    fecha_limite timestamp not null,
    id_evento int not null,
    id_usuario int not null,

    constraint fk_tarea_evento
        foreign key (id_evento)
        references eventos(id_evento),

    constraint fk_tarea_usuario
        foreign key (id_usuario)
        references usuarios(id_usuario)
);

alter table eventos
add column id_ubicacion int references ubicaciones(id_ubicacion) on delete set null;
```

```
create or replace view vista_ocupacion_espacio as
select
    ub.id_ubicacion,
    ub.nombre as ubicacion,
    ub.direccion,
    ub.capacidad,
    (select count(*) from eventos e where e.id_ubicacion=ub.id_ubicacion) as cantidad_eventos
from ubicaciones ub
order by cantidad_eventos desc;
create or replace view vista_disponibilidad_usuarios as
select
    d.id_disponibilidad,
    d.id_usuario,
    u.nombre,
    u.apellido,
    d.fecha,
    d.hora_inicio,
    d.hora_fin,
    d.tipos_disponibilidad
from disponibilidades d, usuarios u
where d.id_usuario = u.id_usuario;
create or replace view vista_tareas_asignadas AS
select
    t.id_tarea,
    u.id_usuario,
    u.nombre AS nombre_usuario,
    u.apellido AS apellido_usuario,
    e.id_evento,
    e.titulo AS evento,
    t.titulo AS tarea,
    t.descripcion,
    t.estado,
    t.prioridad,
    t.fecha_limite
from tareas t, usuarios u, eventos e
where t.id_usuario = u.id_usuario
and t.id_evento = e.id_evento;
```

Justificación del Diseño del Modelo Físico (DDL) y Vistas Analíticas

1. Tabla ubicaciones

- a. Atributo nombre: Se definió un campo atómico varchar(30) que incluye la clasificación descriptiva del espacio (ej. "Sala A", "Recinto B", "Auditorio C"). Se descartó crear una tabla o columna separada para el tipo de lugar para evitar redundancia de datos, ya que los atributos solicitados por requerimiento son id_ubicacion, nombre, direccion, ciudad y capacidad.
- b. Capacidad smallint check (capacidad > 0): Se eligió el tipo smallint (rango hasta 32,767) para optimizar el almacenamiento en disco, considerando que la capacidad de una sala no superará dicho entero. La restricción CHECK impone integridad de dominio a nivel de motor, impidiendo aforos nulos o negativos.

2. Tabla disponibilidades

- a. hora_inicio < hora_fin (chk_horas): Restricción de verificación que garantiza la coherencia temporal de las franjas horarias, evitando que se registren intervalos invertidos o nulos en la base de datos.
- b. tipos_disponibilidad (chk_tipo_dispo): Restricción CHECK que limita los valores a ('Disponible', 'Ocupado', 'No disponible'). Esto asegura la integridad del catálogo sin recurrir a una tabla foránea adicional, optimizando el rendimiento en las consultas de convocatoria.
- c. Tipos de datos date y time: Se desacopló la fecha de la hora para facilitar la evaluación rápida de disponibilidades diarias recurrentes y simplificar las consultas relacionales desde Python.

3. Tabla Tareas

- a. estado y prioridad con DEFAULT y CHECK: Se asignaron valores por defecto ('Pendiente' y 'Media') para agilizar la captura de datos en la GUI. Las restricciones CHECK restringen el ciclo de vida a los estados autorizados (Pendiente, En progreso, Completada, Cancelada) y prioridades (Baja, Media, Alta).
- b. fecha_limite timestamp: Se utilizó timestamp para permitir un control exacto (fecha y hora) al contrastar contra el reloj del servidor (datetime.now()) y determinar de forma precisa si una tarea está vencida.
- c. Claves Foráneas (fk_tarea_evento y fk_tarea_usuario): Garantizan integridad referencial estricta. Cada tarea debe pertenecer obligatoriamente a un evento agendado y estar asignada a un usuario responsable.

4. Alteracion a eventos

- a. ON DELETE SET NULL: Se configuró borrado nulo en la clave foránea hacia ubicaciones. Si una ubicación física se elimina del sistema, la agenda histórica de los eventos se preserva (quedando la ubicación en NULL) en lugar de borrar en cascada los eventos de los usuarios.

Justificación de las Vistas

1. vista_ocupacion_espacio

- a. Objetivo de Diseño: Generar el reporte del ranking de recintos solicitados.
- b. Justificación Técnica: Utiliza una subconsulta escalar para calcular COUNT(e.id_evento) sobre cada ubicación y aplica un ORDER BY cantidad_eventos DESC. Al estar encapsulada en la base de datos, la GUI ejecuta un simple SELECT * de solo lectura, trasladando el esfuerzo de cómputo al motor de PostgreSQL y garantizando actualización en tiempo real cuando se agendan nuevos eventos.

2. vista_disponibilidad_usuarios

- a. Objetivo de Diseño: Presentación limpia del módulo de tiempos.
- b. Justificación Técnica: Realiza la combinación de la tabla disponibilidades con usuarios para exponer el nombre completo de la persona junto a sus franjas. Esto permite que el componente

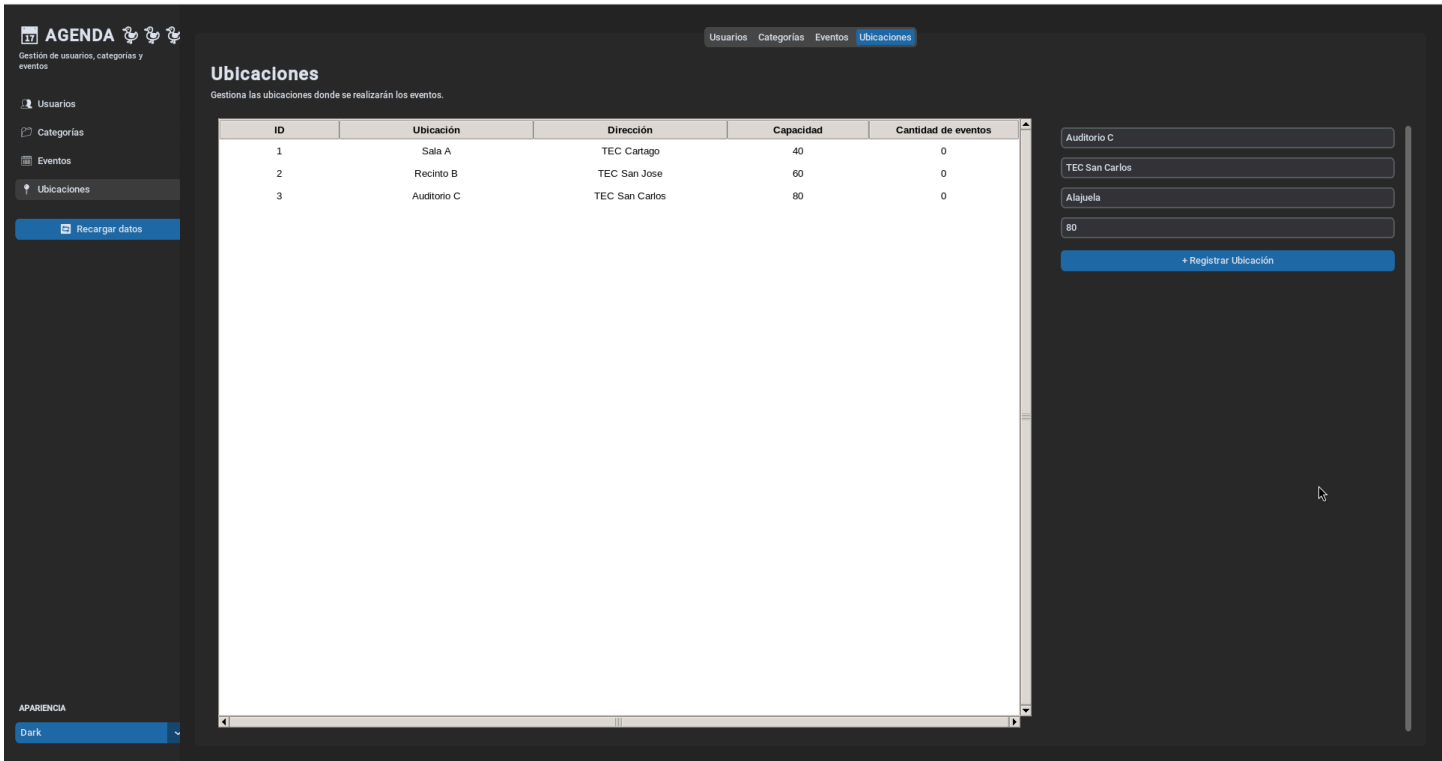
Treeview de Python cargue la información directamente sin construir consultas JOIN complejas en la capa de aplicación.

3. vista_tareas_asignadas

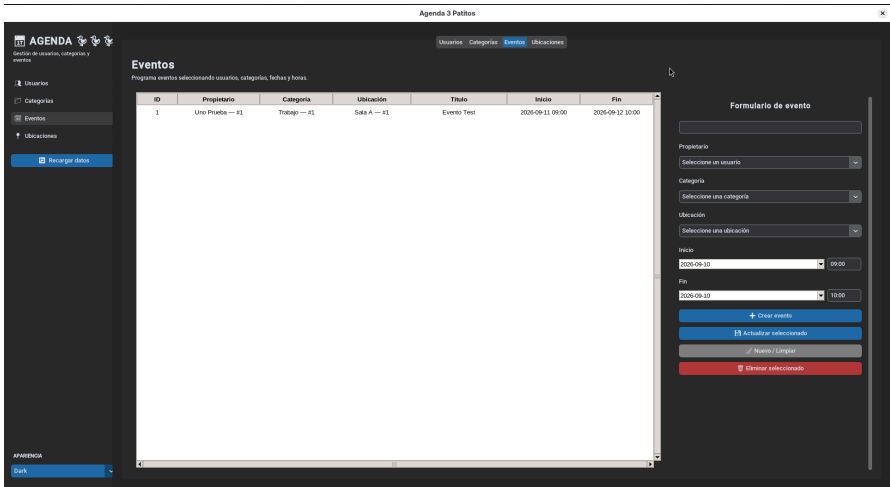
- a. Objetivo de Diseño: Monitoreo y control de la carga de trabajo por usuario.
- b. Justificación: Realiza la proyección de las tablas tareas, usuarios y eventos. Centraliza en un solo recurso relacional el título del evento, el responsable y el estado de la tarea, facilitando la lectura desde la interfaz gráfica para la detección de cuellos de botella y tareas fuera de plazo.

Evidencias Visuales de la Interfaz Gráfica

Módulo Gestor de Ubicaciones



Vista de la tabla con los datos, formulario y botones alineados.



Integración con eventos, formulario habilitado para añadir Ubicaciones

Evidencia proceso y requerimientos del módulo

Tabla de Ubicaciones antes

17

AGENDA

Gestión de usuarios, categorías y eventos

Usuarios

Categorías

Eventos

Ubicaciones

Recargar datos

UsuariosCategoríasEventosUbicaciones

Ubicaciones

Gestiona las ubicaciones donde se realizarán los eventos.

ID	Ubicación	Dirección	Capacidad	Cantidad de eventos
1	Sala A	TEC Cartago	40	1
2	Recinto B	TEC San Jose	60	0
3	Auditorio C	TEC San Carlos	80	0

Nombre (ej. Sala A)

Dirección

Ciudad

Capacidad

+ Registrar Ubicación

Creamos un evento y le añadimos Ubicación

17

AGENDA

Gestión de usuarios, categorías y eventos

Usuarios

Categorías

Eventos

Ubicaciones

Recargar datos

UsuariosCategoríasEventosUbicaciones

Eventos

Programa eventos seleccionando usuarios, categorías, fechas y horas.

ID	Propietario	Categoría	Ubicación	Título	Inicio	Fin
1	Uno Prueba — #1	Trabajo — #1	Sala A — #1	Evento Test	2026-09-11 09:00	2026-09-12 10:00
2	Uno Prueba — #1	Trabajo — #1	Recinto B — #2	Evidencia Modulo 1	2026-09-21 07:00	2026-09-21 12:00
1	Uno Prueba — #1	Trabajo — #1	Sala A — #1	Evento Test	2026-09-11 09:00	2026-09-12 10:00

Éxito

Evento creado correctamente.

OK

Formulario de evento

Título del evento

Propietario

Seleccione un usuario

Categoría

Seleccione una categoría

Ubicación

Recinto B - #2

Inicio

2026-09-1009:00

Fin

2026-09-1010:00

+ Crear evento

Actualizar seleccionado

Nuevo / Limpiar

Eliminar seleccionado

Tabla de Ubicaciones después

UsuariosCategoríasEventosUbicaciones

Ubicaciones

Gestiona las ubicaciones donde se realizarán los eventos.

ID	Ubicación	Dirección	Capacidad	Cantidad de eventos
1	Sala A	TEC Cartago	40	1
2	Recinto B	TEC San Jose	60	1
3	Auditorio C	TEC San Carlos	80	0

Nombre (ej. Sala A)

Dirección

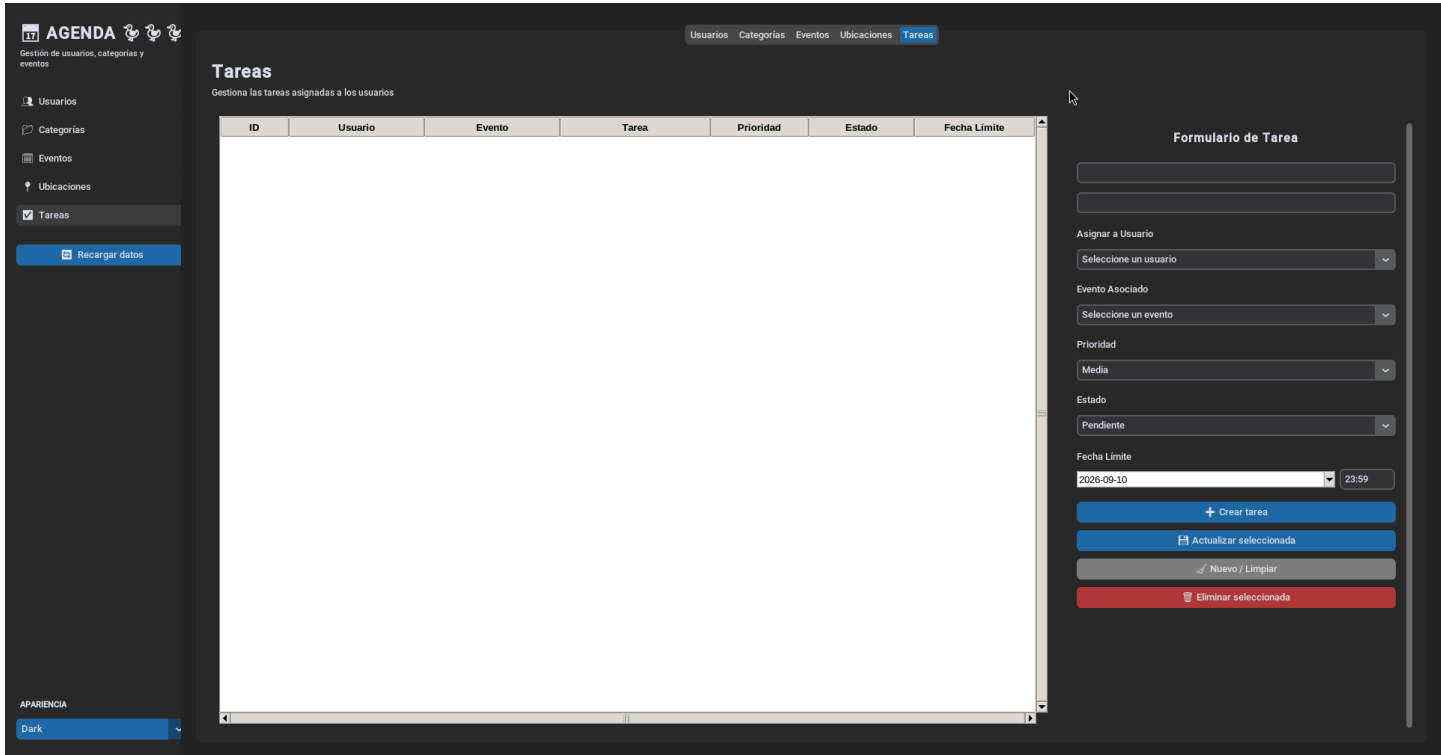
Ciudad

Capacidad

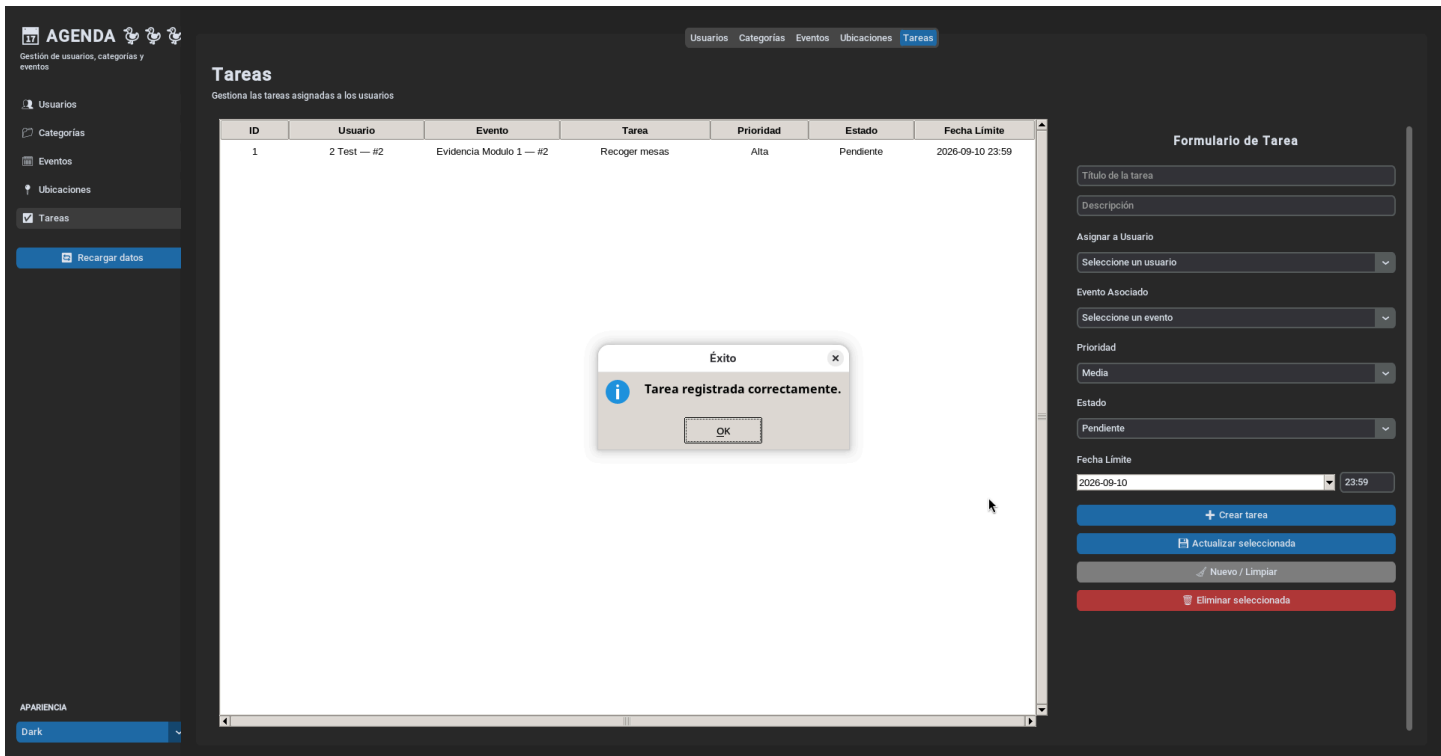
+ Registrar Ubicación

Se agrega y suma la pestaña de eventos

Módulo Tareas Asociadas a Eventos



Interfaz de Tareas con formulario, desplegable y tabla de seguimiento.



Registro de nueva tarea con vinculación hacia eventos y usuarios.

UsuariosCategoríasEventosUbicacionesTareas

Tareas

Gestiona las tareas asignadas a los usuarios

ID	Usuario	Evento	Tarea	Prioridad	Estado	Fecha Límite
2	Uno Prueba — #1	Mejenga — #3	Test Vencida	Media	▲ VENCIDA (Pendien	2026-09-09 23:59
1	2 Test — #2	Evidencia Modulo 1 — #2	Recoger mesas	Alta	Pendiente	2026-09-10 23:59

(En el momento del registro de esta consulta es 10 de Septiembre) Si la fecha límite es anterior al hoy, sola se registra como Vencida con el estado que le pongamos.

UsuariosCategoríasEventosUbicacionesTareas

Tareas

Gestiona las tareas asignadas a los usuarios

ID	Usuario	Evento	Tarea	Prioridad	Estado	Fecha Límite
2	Uno Prueba — #1	Mejenga — #3	Test Vencida	Media	▲ VENCIDA (En progreso)	2026-09-09 23:59
1	2 Test — #2	Evidencia Modulo 1 — #2	Recoger mesas	Alta	Pendiente	2026-09-10 23:59

Si le actualizamos el estado igual seguira Vencida, solo colocando como Estado “Completada” deja de verse Vencida

UsuariosCategoríasEventosUbicacionesTareas

Tareas

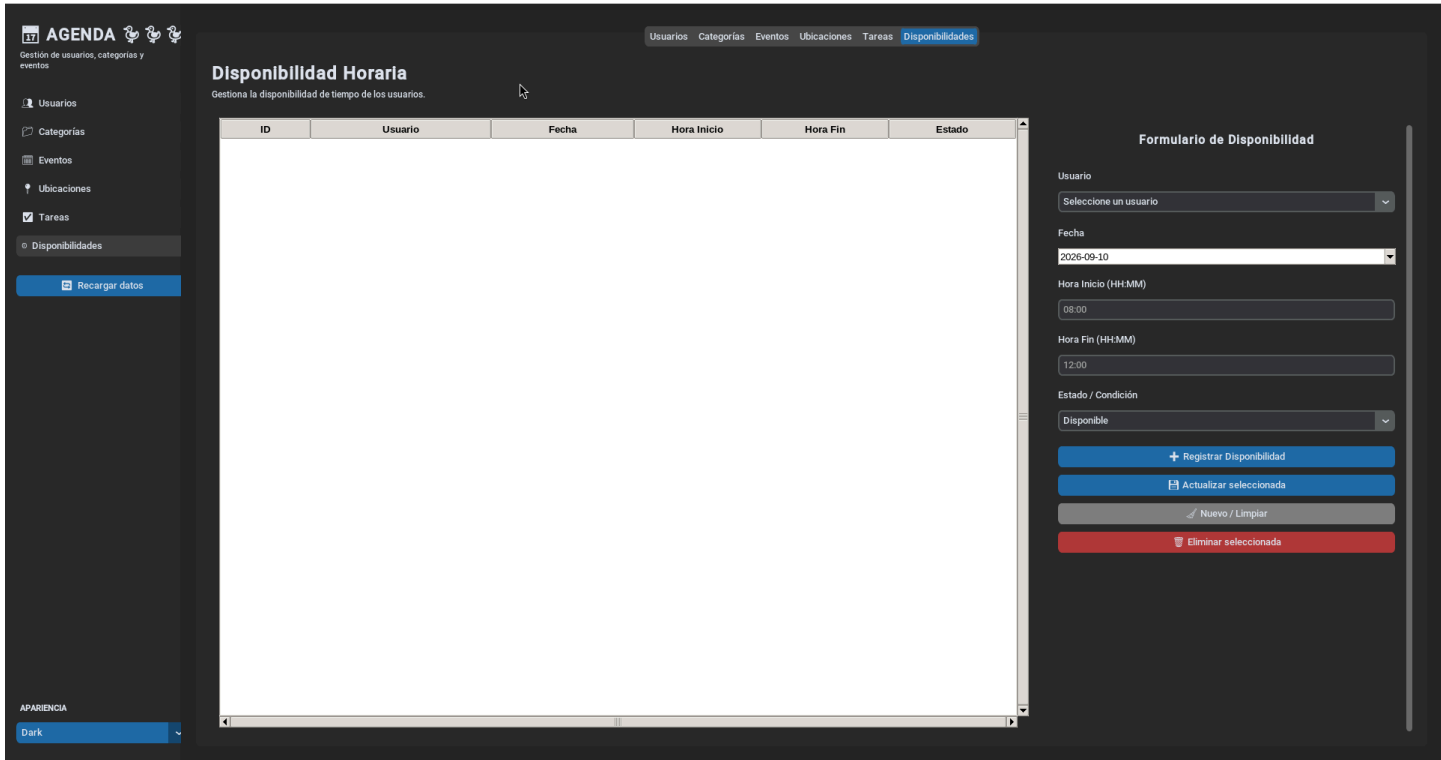
Gestiona las tareas asignadas a los usuarios

ID	Usuario	Evento	Tarea	Prioridad	Estado	Fecha Límite
2	Uno Prueba — #1	Mejenga — #3	Test Vencida	Media	Completada	2026-09-09 23:59
1	2 Test — #2	Evidencia Modulo 1 — #2	Recoger mesas	Alta	Pendiente	2026-09-10 23:59

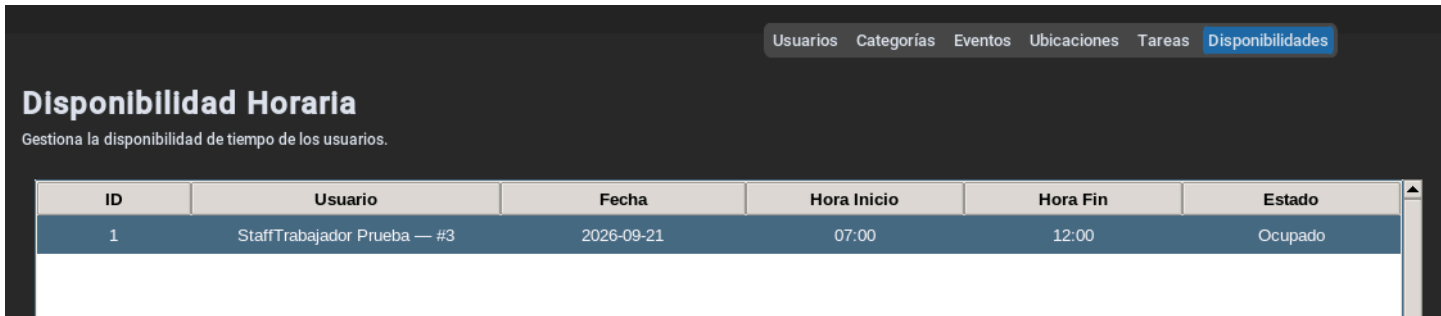
Esto cumple los requerimientos funcionales asignados a este módulo:

- **RF-15 (Asignación de Tareas):** Registro y consulta relacional de actividades vinculadas dinámicamente a eventos y usuarios específicos.
- **RF-16 (Control Visual de Vencimientos):** Evaluación dinámica de la fecha_limite en la capa de aplicación. El sistema marca automáticamente el estado ⚠ VENCIDA si la fecha expira, conservando el estado base (Pendiente o En progreso) y removiendo la alerta únicamente cuando la tarea alcanza el estado Completada.
- **RF-17 (Consulta Consolidada):** Despliegue optimizado de datos utilizando la vista vista_tareas_asignadas en PostgreSQL para garantizar la integridad y rendimiento en la presentación del Treeview.

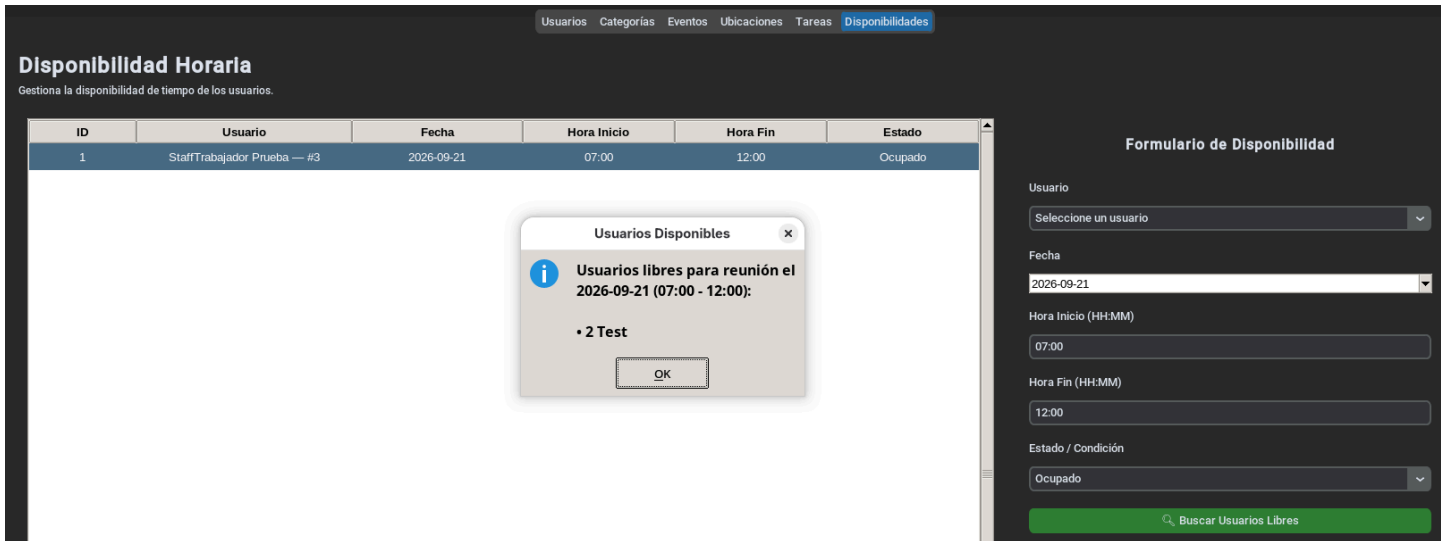
Módulo Disponibilidad de Usuarios y Gestión de Tiempos



Interfaz Disponibilidades con tabla y formulario con botones y desplegables



Registramos una tarea para observar el comportamiento



Si buscamos la fecha en el periodo de la tarea registrada, nos aparece únicamente un usuario de tres, StaffTrabajador está registrado como ocupado y el otro usuario “Uno Prueba” hace un evento ese día a esa hora.

Eventos

Programa eventos seleccionando usuarios, categorías, fechas y horas.

ID	Propietario	Categoría	Ubicación	Título	Inicio	Fin
2	Uno Prueba — #1	Trabajo — #1	Recinto B — #2	Evidencia Modulo 1	2026-09-21 07:00	2026-09-21 12:00

Disponibilidad Horaria

Gestiona la disponibilidad de tiempo de los usuarios.

ID	Usuario	Fecha	Hora Inicio	Hora Fin	Estado
1	StaffTrabajador Prueba — #3	2026-09-21	07:00	12:00	Ocupado

Conflicto de Horario

El usuario ya tiene un registro de disponibilidad u ocupación que se solapa con este rango de tiempo.

OK

Formulario de Disponibilidad

Usuario

StaffTrabajador Prueba — #3

Fecha

2026-09-21

Hora Inicio (HH:MM)

08:00

Hora Fin (HH:MM)

12:00

Estado / Condición

Disponible

Buscar Usuarios Libres

Registrar Disponibilidad

Si Intentamos registrar disponibilidad en un espacio ya ocupado tenemos un conflicto y no se logra registrar, así solucionamos un solapamiento

Disponibilidad Horaria

Gestiona la disponibilidad de tiempo de los usuarios.

ID	Usuario	Fecha	Hora Inicio	Hora Fin	Estado
1	StaffTrabajador Prueba — #3	2026-09-21	07:00	12:00	Ocupado

Conflicto con Evento

El usuario no puede marcarse como 'Disponible' porque ya tiene agendado el evento 'Evidencia Modulo 1' en ese mismo horario.

OK

Formulario de Disponibilidad

Usuario

Uno Prueba — #1

Fecha

2026-09-21

Hora Inicio (HH:MM)

08:00

Hora Fin (HH:MM)

12:00

Estado / Condición

Disponible

Buscar Usuarios Libres

Registrar Disponibilidad

De igual forma si intentamos registrar una tarea a un usuario que ya tiene asignado un evento,el sistema bloquea el solapamiento e impide registrarlo.

Anexo IA

Se utilizó el asistente de IA (Gemini) como herramienta de apoyo pedagógico y técnico para acelerar el aprendizaje del lenguaje Python (librerías CustomTkinter y psycpg2), así como para apoyar en la lógica de consultas relacionales en PostgreSQL. La autoría, diseño de la base de datos y validación de las pruebas corresponden íntegramente al estudiante.

Módulo	Componente en que se usó IA	Apoyo Específico de Python / SQL
Ubicaciones	GUI y Vista Analítica	Sintaxis para la tabla Treeview en CustomTkinter y subconsulta COUNT en vista_ocupacion_espacio.
Eventos	Integración Relacional	Manejo de eventos en widgets combo (ComboBox) y casteo de cadenas de texto a TIMESTAMP en Python.
Tareas	Control de Vencimientos	Uso de la librería datetime para comparar fechas contra el reloj del sistema y sintaxis del Treeview.
Disponibilidades	Algoritmo de Convocatoria	Estructuración de consultas con NOT EXISTS y casteo explícito (::time / ::timestamp) para detectar solapamientos.